Team notebook

March 27, 2026

Contents

1 00-Template

1.1 template

```cpp
#include<bits/stdc++.h>
using namespace std;
//tipos
using ll = long long;
using ld = long double;
using pii = pair<int,int>;
using pll = pair<ll,ll>;
//loops
#define FOR(i,n) for(int i=0; i<(int)(n); i++)
#define FOR1(i,n) for(int i=1; i<=(int)(n); i++)
#define FORA(x,a) for(auto &x : a)
//Sorts
#define all(x) begin(x), end(x)
#define rall(x) rbegin(x), rend(x)
//debug
#define dbg(x) cerr << #x << " = " << x << "\n"

//  ll aux = LLONG_MAX;
//  int a = INT_MAX;

int main(){
    ios::sync_with_stdio(false);cin.tie(0);
    int n = 3;
    dbg(n);
    return 0;
}
```

2 01-Buscas

2.1 1-BinariaTernaria

```cpp
#include<bits/stdc++.h>
using namespace std;
using ll = long long;

void bin(){
    int n,k; cin>>n>>k;
    vector<int> vec;
    int l = 0; int r = n-1;
    int mid;
    while(r >= l){
        mid = l + ((r-l)/2);
        if(vec[mid] > k){
            r = mid - 1;
        }
        else if(vec[mid] <= k){
            l = mid + 1;
        }
    }
}

double f(double x) { // func unimodal
    return -(x * x) + 4 * x;
}
double ternarySearchDouble(double l, double r) {
    for (int i = 0; i < 200; i++) {
        double m1 = l + (r - l) / 3.0;
        double m2 = r - (r - l) / 3.0;
        if (f(m1) < f(m2)) {
            l = m1;
        } else {
            r = m2;
        }
    }
    return f(l);
}
ll f2(ll x) {
    return -(x * x) + 4 * x;
}
ll ternarySearchInt(ll l, ll r) {
    while (r - l > 2) {
        ll m1 = l + (r - l) / 3;
        ll m2 = r - (r - l) / 3;
        if (f2(m1) < f2(m2)) {
            l = m1;
        } else {
            r = m2;
        }
    }
    ll ans = f2(l);
    for (ll i = l + 1; i <= r; i++) {
        ans = max(ans, f2(i));
    }
    return ans;
}
```

2.2 2-TernariaAninhada

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
/*
vector<Point> vacas;

double f(double x, double y) {
    double resposta = 0.0;
    double aux; Point aux2;
    for(int i = 0; i<n; i++){
        aux2.x = x; aux2.y = y; aux2.raio = vacas[i].raio;
        aux = dist(vacas[i], aux2) - aux2.raio;
        if(aux < 0.0) continue;
        if(resposta - aux < eps){
            resposta = aux;
        }
    }
    return resposta;
}
double ternaryY(double x) {
    double l = -1000, r = 1000;
    for (int i = 0; i < 100; i++) { //limitar quantidade de
         vezes da procura, util para ld, ou double
        double m1 = l + (r - l) / 3.0;
        double m2 = r - (r - l) / 3.0;
        if (f(x, m1) > f(x, m2)) {
            l = m1;
        } else {
            r = m2;
        }
    }
    return f(x, l);
}
double ternaryX() {
    double l = -1000, r = 1000;
    for (int i = 0; i < 100; i++) {
        double m1 = l + (r - l) / 3.0;
        double m2 = r - (r - l) / 3.0;
        if (ternaryY(m1) > ternaryY(m2)) {
            l = m1;
        } else {
            r = m2;
        }
    }
    return ternaryY(l);
} //exemplo de ternria aninhada
 */

// while(r - l > 2){
//     int m1 = l + ((r-l)/3);
//     int m2 = r - ((r-l)/3);
//     r1 = custoE[m1] + dist(right[i], left[m1]);
//     r2 = custoE[m2] + dist(right[i], left[m2]);
//     if(r1 < r2){
//         r = m2;
//     }
//     else{
//         l = m1;
//     }
// } // escopo
```

3 02-CaminhosMinimosGrafos

3.1 bellmanFord

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
#define FOR(i,n) for(int i=0; i<(int)(n); i++)
const int n = 1e5;
vector<ll> dist (n, LLONG_MAX);
void bellman_Ford(){
    dist[1] = 0;
    vector<tuple<int,int,int>> edges;
    //ler as arestas e pesos respectivos.
    //interessante botar um supervertice 0 que chega em todos
         com distancia 0.
    //pode - se usar um vetor de pais para salvar o caminho do
         feito, mas antes de printar o caminho se atualiza n
         vezes o pai de v para garantir que ele esta dentro do
         ciclo.
    //Tambem   plausivel uma segunda iterao para tentar
         repassar o status de relaxamento, com o intuito de
         propagar os vertices de ciclos negativos e ver se eles
         alcanam o vertice de saida.
    FOR(rep,n-1){
        for(auto [u,v,w]: edges){
            dist[v] = min(dist[v], dist[u] + w);
        }
    }
    //verificar se tem um ciclo negativo: (iterar depois de
         construir o vetor de dist anterior).
    bool ciclonegativo = false;
    for(auto [u,v,w]: edges){
        if(dist[v] > dist[u] + w){
            ciclonegativo = true;
            break;
        }
    }

}
```

3.2 dijkstra

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
using pll = pair<ll,ll>;
const int n = 1e5 + 10;
// no  funciona com arestas negativas
//pode ser multisource
void dijkstra(){
    vector<ll> dist (n+1, LLONG_MAX);
    vector<vector<pll>> adj (n+1);
    //preencher adj com as entradas das arestas e pesos.
    priority_queue<pll, vector<pll>, greater<pll>> pq;
    pq.push({0,1});
    dist[1] = 0;
    while(!pq.empty()){
        auto [d, v] = pq.top();
        pq.pop();
        if(d<=dist[v]){
            for(auto [u,w]: adj[v]){
                if(dist[u] > dist[v] + w){
                    dist[u] = dist[v] + w;
                    pq.push({dist[u], u});
                }
            }
        }
    }

}
```

3.3 floydWarshall

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
#define FOR1(i,n) for(int i=1; i<=(int)(n); i++)
const int n = 300 + 10;
void floyd_warshall(){
    vector<vector<ll>> dist (n+1, vector<ll> (n+1));
    //ao preencher multiplas arestas eu so pego o minimo
    //dist[i][j] com i == j deve ser 0;
    FOR1(k,n){
        FOR1(u,n){
            FOR1(v,n){
                if(dist[u][k] != LLONG_MAX && dist[k][v] !=
                     LLONG_MAX){
                    dist[u][v] = min(dist[u][v], dist[u][k] +
                         dist[k][v]);
                }
            }
        }
    }
}
```

4 03-DSUandTOPOSORT

4.1 dsu

```
#include<bits/stdc++.h>
using namespace std;
```

```cpp
using ll = long long;
#define FOR(i,n) for(int i=0; i<(int)(n); i++)
int const n = 1e5;
int pai[n];
int sz[n];

int find(int u){
    if(u == pai[u]) return u;
    return pai[u] = find(pai[u]);
}

void join(int u, int v){
    u = find(u);
    v = find(v);

    if(u == v) return;
    if(sz[u]<sz[v]) swap(u,v);

    pai[v] = u;
    sz[u] = sz[u] + sz[v];
}

int main(){
    ios::sync_with_stdio(false);cin.tie(0);
    FOR(i,n){
        pai[i] = i;
        sz[i] = 1;
    }
    return 0;
}
```

4.2 toposort

```cpp
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
using pii = pair<int,int>;
const int MAXN = 1e5 + 1;

vector<int> adj[MAXN];
int deg[MAXN];
int n;

vector<int> toposort(vector<pii>&edges){
    memset(deg, 0, sizeof(deg));
    for(auto [u, v]: edges){
        deg[v]++;
    }
    queue<int>q;
    for(int i = 1; i <= n; i++){
        if(deg[i] == 0) q.push(i);
    }
    vector<int>ord;
    while(!q.empty()){
        int u = q.front(); q.pop();
        ord.push_back(u);
        for(int v: adj[u]){
            deg[v]--;
            if(deg[v] == 0) q.push(v);
        }
    }
    // se ord.size() != n, no h   ordenao   possvel !
    return ord;
}
```

5 04-TeoriaDosNumeros

5.1 matematica

```cpp
#include<bits/stdc++.h>
using namespace std;
using ll = long long;

const int n = 2e5 + 1;
const int MOD = 1e9 + 7;

vector<vector<int>> primos (n);
vector<bool> visitados (n, false);

vector<int> getdivisors(int n) {
    vector<int> divisors;
    for(int i = 1; i * i <= n; i++){
        if(n % 1 == 0){
            divisors.push_back(i);
            if(i != n/i) divisors.push_back(n/i);
        }
    }
    return divisors;
}

const int m = 1e7;
bool isPrime[m];

vector<int> crivo(){ // calcular todos os primos
    memset(isPrime, 1, sizeof(isPrime));
    isPrime[0] = isPrime[1] = false;
    vector<int> primes;
    for(int i = 2; i < m; i++){
        if(isPrime[i]){
            primes.push_back(i);
            for(int j = i + i; j < m; j += i){
                isPrime[j] = false;
            }
        }
    }
    return primes;
}

void sieve(){ // calcula fatoracao prima de numeros (so
     numeros, sem quantidade)
    for(int i = 2; i<=n-1; i++){
        if(visitados[i]) continue;
        visitados[i] = true;
        for(int j = i; j<=n-1; j+=i){
            visitados[j] = true;
            primos[j].push_back(i);
        }
    }
}

void fact(ll& x){
    ll res = 1;
    for(int i = 2; i<= x; i++){
        res *= i;
        res = res % MOD;
    }
    x = res;
}

ll mod_add(ll a, ll b){
    return (a % MOD + b % MOD) % MOD;
}
ll sub(ll a, ll b){
    return((a % MOD - b % MOD) + MOD) % MOD;
}
ll mod_mul(ll a, ll b){
    return(a % MOD * b % MOD) % MOD;
}

ll fexp(ll a, ll b){ // num a, primo b - 2 (1e9 + 7 - 2)
    ll ret = 1;
    while(b){
        if(b%1) ret = mod_mul(ret,a);
        a = mod_mul(a,a);
        b >>= 1;
    }
    return ret % MOD;
}

long double log7(ll w){ //trocar base
    return log(w)/log(7.0);
}
```

6 05-DP

6.1 bitTricks

```cpp
#include<bits/stdc++.h>
using namespace std;
using ll = long long;

// ligar i esimo (n |= (1<<i))
// testar i esimo (n & (1<<i))
// flipar i esimo (n ^= (1<<i))
// testar se e pot de 2 !(n & (n-1))
// contar quantos bits estao ligados __builtin_popcount(n)
// testar cheio ((1<<n) -1)
// lsb (n & -n)
```

```
int tsp(){
    int n; cin>>n;
    vector<int> vec (20,0);
    vector<vector<int>> dp (1<<20, vector<int> (20,0));
    int aux;
    for(int i=0;i<n;i++){
        cin>>aux;
        vec[aux-1]++;
    }
    for(int i = 1; i<(1<<20); i++){
        for(int j = 0; j<20; j++) if((i & (1<<j)) && vec[j]>0){
            for(int k = 0; k<20; k++) if(!(i & (1<<k)) && vec[k]
                  > 0){
                dp[i | (1<<k)][k] = max(dp[i | (1<<k)][k],
                     dp[i][j] + gcd(j+1,k+1));
            }
        }
    }
    int resposta = 0;
    int maior = 0;
    for(int i = 0;i<20;i++){
        if(vec[i]!=0){
            maior = max(maior, dp[(1<<20)-1][i]);
            if(vec[i]>1){
                resposta += (i+1)*(vec[i]-1);
            }
        }
    }
    resposta += maior;
    return resposta;
} //exemplo de tsp
```

6.2 dp

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;

const int t = 1e5;

int lis(){
    int answer = 0;
    vector<int> dp(t);
    for (int j = 0; j < t; ++j) {
        dp[j] = 1;
        for (int i = j - 1; i >= 0; --i) {
            if (a[i] < a[j]) {
                dp[j] = max(dp[j], dp[i] + 1);
            }
        }
        answer = max(answer, dp[j]);
    }
    return answer;
}

string a, b;
vector<vector<int>> dyp;

int best(int i, int j){
    if (i < 0 or j < 0) return 0;
    if (dyp[i][j] != -1) return dyp[i][j];

    dyp[i][j] = 0;
    if (a[i] == b[j]) {
        dyp[i][j] = 1 + best(i - 1, j - 1);
    } else {
        dyp[i][j] = max(best(i - 1, j), best(i, j - 1));
    }
    return dyp[i][j];
}

int lcs(){
    int n, m;
    cin>>a; cin>>n;
    cin>>b; cin>>m;
    dyp = vector<vector<int>> (n,vector<int>(m,-1));
    return best(n-1, m-1);
}

bool subsetSum() {
    int t = 1;
    vector<int> a = {3, 4, 7, 2};
    vector<bool> dp(t + 1);
    dp[0] = true;
    for (int x : a) {
        for (int i = t - x; i >= 0; --i) {
            if (dp[i]) {
                dp[i + x] = true;
            }
        }
    }
    return dp[t];
}

int knap() {
    int n = 4, W = 7;
    vector<int> w = {3, 4, 2, 6};
    vector<int> v = {4, 5, 3, 7};
    vector<int> knapsack(W + 1);
    for (int i = 0; i < n; ++i) {
        for (int k = W - w[i]; k >= 0; --k) {
            knapsack[k + w[i]] = max(knapsack[k + w[i]],
                 knapsack[k] + v[i]);
        }
    }
    return knapsack[W];
}
```

7 06-DANDC

7.1 DANDC

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
struct Query{
    int l, r, idx;
};
vector<int> vec;
vector<int> resposta;
vector<Query> queries;
vector<Query> aux;
vector<int> s, p;
void dnc(int l, int r, int ql, int qr){
    if(l > r || ql > qr) return;
    int m = l + (r-l)/2;
    int cnt_l = 0, cnt_m = 0, cnt_r = 0; // quantidade de
         queries q ficam a esquerda, tm ou ficam a direita de m
    for(int i = ql; i <= qr; i++){
        if(queries[i].l <= m && m <= queries[i].r) cnt_m++;
        else if(queries[i].r < m) cnt_l++;
        else cnt_r++;
    }
    int p_l = ql, p_m = ql + cnt_l, p_r = p_m + cnt_m; //
         iterador que indica inicio de cada uma desses grupos
         [esquerda,m,direita]
    for(int i = ql; i <= qr; i++){ //atualiza aux para
         delimitar [esquerda,meio,direita]
        if(queries[i].l <= m && queries[i].r >= m){
            aux[p_m] = queries[i];
            p_m++;
        }
        else if(queries[i].r < m){
            aux[p_l] = queries[i];
            p_l++;
        }
        else{
            aux[p_r] = queries[i];
            p_r++;
        }
    }
    for(int i = ql; i <= qr; i++){ //queries pega os valores
         antes organizados
        queries[i] = aux[i];
    }
    int l_start = ql, l_end = ql + cnt_l - 1; // indicadores
         comeco e fim de cada um dos grupos
         [esquerda,meio,direita]
    int m_start = l_end + 1, m_end = m_start + cnt_m - 1;
    int r_start = m_end + 1, r_end = qr;
    dnc(l, m - 1, l_start, l_end);
    dnc(m+1, r, r_start, r_end);
    if(m_start > m_end) return;
    s[m] = vec[m]; //salvando menor sufixo [l,m]
    for(int i = m-1; i>=l; i--){
        s[i] = min(vec[i], s[i+1]);
    }
    p[m] = vec[m]; // salvando menor prefixo [m,r]
    for(int i = m+1; i<=r; i++){
        p[i] = min(vec[i], p[i-1]);
    }
    Query x;
    for(int i = m_start; i<= m_end; i++){
```

```
        x = queries[i];
        resposta[x.idx] = min(s[x.l], p[x.r]); //query x  uma
             query do meio que tem como menor valor ou o menor
             sufixo [l,m] ou o menor prefixo [m,r]
    }
}

void solve(){
    int n, q; cin>>n>>q;
    vec = vector<int> (n);
    s = vector<int> (n);
    p = vector<int> (n);
    resposta = vector<int> (q);
    queries = vector<Query> (q);
    aux = vector<Query> (q);
    for(int i=0;i<n;i++) cin>>vec[i];
    for(int i=0;i<q;i++){
        cin>>queries[i].l;
        cin>>queries[i].r;
        queries[i].l--; queries[i].r--;
        queries[i].idx = i;
    }
    dnc(0, n-1, 0, q-1);
    for(int i = 0;i<q;i++){
        cout<<resposta[i]<<'\n';
    }
}
```

7.2 mergeSort

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
vector<int> vec;
vector<int> aux;
ll inversoes = 0;
void merge(int l, int m, int r){
    int i = l, j = m+1, k = l;
    while(i <= m && j <= r){
        if(vec[i] < vec[j]){
            aux[k] = vec[i];
            i++;
        }
        else{
            aux[k] = vec[j];
            j++;
        }
        k++;
    }
    while(i <= m){
        aux[k] = vec[i];
        i++;
        k++;
    }
    while(j <= r){
        aux[k] = vec[j];
        j++;
        k++;
    }
    return;
}
void mergeSort(int l, int r){
    if(l >= r) return;
    int m = l + (r-l)/2;
    mergeSort(l,m);
    mergeSort(m+1,r);
    merge(l, m, r);
    ll a = 0;
    for(int i = l; i<=m; i++){
        while(a < (r - m) && vec[a + m + 1] <= vec[i]) a++;
        inversoes += a;
    }
    for(int i = l; i<= r; i++){
        vec[i] = aux[i];
    }
}
void solve(){
    int n; cin>>n;
    vec = vector<int> (n);
    aux = vector<int> (n);
    for(int i = 0;i<n; i++){
        cin>>vec[i];
    }
    mergeSort(0, n-1);
    for(int i = 0; i<n; i++){
        cout<<vec[i]<<" ";
    }
    cout<<'\n';
    cout<<inversoes;
}
```

8 07-GeometriaComputacional

8.1 gTemplate

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;

const double eps = 1e-12;

int cmp(double a, double b){ // mudar cmp tambem
    if(abs(a-b) < eps) return 0;
    if(a < b) return -1;
    else return 1;
}

struct Point{ //mudar tipo dependendo da questo
        double x, y;
        Point(double x = 0, double y = 0) : x(x),y(y){}
        Point(const Point& p): x(p.x), y(p.y){}
    bool operator < (const Point &p) const{
        if(cmp(x, p.x) != 0) return x < p.x;
        return cmp(y, p.y) < 0;
    }
    bool operator == (const Point &p) const{return !cmp(x, p.x)
         && !cmp(y, p.y);}
    bool operator != (const Point &p) const{return !(p ==
         *this);}

        Point operator + (const Point &p) const{return Point(x
             + p.x, y + p.y);}
        Point operator - (const Point &p) const{return Point(x
             - p.x, y - p.y);}
        Point operator * (const double k) const{return
             Point(x*k,y*k);}
        Point operator / (const double k) const{return
             Point(x/k,y/k);}
        Point& operator=(const Point&) = default;
};

double dot(const Point& p, const Point& q){return ((p.x * q.x)
     + (p.y * q.y));}
long double cross(const Point& p, const Point& q){return ((p.x
     * q.y) - (p.y * q.x));} // (modulo) area do paralelogramo;
double norm(const Point& p){return hypot(p.x,p.y);}
double dist(const Point& p, const Point& q){return hypot((p.x -
     q.x), (p.y - q.y));}
double dist2(const Point& p, const Point& q){return
     dot(p-q,p-q);}
Point normalize(const Point& p){return p/hypot(p.x,p.y);}
     //retorna vetor unitario;
double angle(const Point& p, const Point& q){return
     atan2(cross(p,q), dot(p,q));}
double angle(const Point& p){return atan2(p.y,p.x);}

vector<Point> convexHull(vector<Point>& pts, bool sorted =
     false){
    int n = pts.size();
    if(!sorted){
        sort(pts.begin(),pts.end());
    }
    vector<Point> lower(n+1), upper(n+1);
    int s = 0;
    for(int i = 0;i < n;i++){
        lower[s++] = pts[i];
        while(s>=3){
            Point a = lower[s-3]; Point b = lower[s-2]; Point c
                 = lower[s-1];
            Point v1 = b-a, v2 = c - b;
            if(cross(v1,v2) > 0){
                break;
            }
            lower[s-2] = lower[s-1];
            s--;
        }
    }
    lower.resize(s);
    s = 0;
    for(int i = 0;i < n;i++){
        upper[s++] = pts[i];
        while(s>=3){
```

```
            Point a = upper[s-3]; Point b = upper[s-2]; Point c
                 = upper[s-1];
            Point v1 = b-a, v2 = c - b;
            if(cross(v1,v2) < 0){
                break;
            }
            upper[s-2] = upper[s-1];
            s--;
        }
    }
    upper.resize(s-1);
    reverse(upper.begin(),upper.end());
    upper.pop_back();
    lower.insert(lower.end(),upper.begin(),upper.end());
    return lower;
}

bool isInside(const vector<Point> &hull, Point pt){
    int n = hull.size();
    Point v0 = pt - hull[0], v1 = hull[1] - hull[0], v2 =
         hull[n-1] - hull[0];
    if(cross(v0,v1) > 0 || cross(v0,v2) < 0){
        return false;
    }
    int l = 1, r = n-1;
    while(l != r){
        int mid = (l + r + 1)/2;
        Point v1 = hull[mid] - hull[0];
        if(cross(v0,v1) < 0){
            l = mid;
        }
        else{
            r = mid - 1;
        }
    }
    v0 = hull[(l+1)%n] - hull[l];
    v1 = pt - hull[l];
    return cross(v0,v1) >= 0;
}
```

9 09-HashingandTrie

9.1 hashingT

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
// A ideia principal  representar os valores da string como uma
     soma dos valores assossiados a cada caractere
// Hash(s) = (somtorio(i = 0 -> i = n-1) Si * k^(n-1-i)) % mod
     M; M e k primos.
// Como tratar colises? dois valores darem um Mesmo Hash? Esse
     caso j  muito improvavel (1/M), mas existe Hash duplo!
// Hash simples M com 100.000 itens j fica muito provvel.
static const ll P1 = 31;
static const ll P2 = 37;
static const ll M1 = 1e9 + 33;
static const ll M2 = 1e9 + 93;

const int MAXN = 100005; //tamanho maximo da string;
ll powers1[MAXN], powers2[MAXN];
ll prefix_hash1[MAXN], prefix_hash2[MAXN];

void calc_powers(int n) { //chamar antes de qualquer coisa
    powers1[0] = 1;
    powers2[0] = 1;
    for (int i = 1; i <= n; i++) {
        powers1[i] = (powers1[i - 1] * P1) % M1;
        powers2[i] = (powers2[i - 1] * P2) % M2;
    }
}

struct StringHash {
    vector<ll> prefix_hash1, prefix_hash2;
    StringHash(const vector<ll>& s) {
        int n = s.size();
        prefix_hash1.assign(n + 1, 0);
        prefix_hash2.assign(n + 1, 0);

        for(int i = 0; i < n; i++){
            ll val = s[i];
            prefix_hash1[i+1] = (prefix_hash1[i] * P1 + val) %
                 M1;
            prefix_hash2[i+1] = (prefix_hash2[i] * P2 + val) %
                 M2;
        }
    }
    pair<ll,ll> query(int i, int j) {
        int len = j - i + 1;
        ll h1 = (((prefix_hash1[j+1] - (prefix_hash1[i] *
             powers1[len]) % M1) % M1) + M1) % M1;
        ll h2 = (((prefix_hash2[j+1] - (prefix_hash2[i] *
             powers2[len]) % M2) % M2) + M2) % M2;
        return {h1, h2};
    }
};

//Hash de
     Sets--------------------------------------------------------------
mt19937_64
     rng(chrono::steady_clock::now().time_since_epoch().count());
//mt19937
     rng((int)chrono::steady_clock::now().time_since_epoch().count());

//pode gerar ints ou doubles, basta mudar o parametro do
     template
long long uniform(long long l, long long r){
    uniform_int_distribution<long long> uid(l, r);
    return uid(rng);
}

void setHash(){
    int n;
    vector<ll> input;
    map<long long, long long> valor;
    for(auto num : input){
        if(!valor.count(num)){
            valor[num] = uniform(1, 1e18);
        }
    }

    long long hsh = 0;
    for(int i = 0; i < n; i++){
        hsh ^= valor[input[i]];
    }
}
//---------------------------------------------------------------------
//pi[i] e o tamanho do maior prefix da substring (0,i) que bate
     perfeitamente
//com o sufixo da mesma string
vector<int> Pi(string &t){
    vector<int> p(t.size(), 0);

    for(int i=1, j=0; i<t.size(); i++){
        while(j > 0 && t[j] != t[i]) j = p[j-1];
        if(t[j] == t[i]) j++;
        p[i] = j;
    }
    return p;
}
```

10 10-SegTreeandFenwickTree

10.1 1-segtree

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;

vector<int> tree;
vector<int> vec;

int join(int l, int r){
    return min(l,r);
}

//buildando segtree
void build(int v, int tl, int tr){
    if(tl == tr){
        tree[v] = vec[tl];
    }
    else{
        int tm = (tl + tr)/2;
        build(2*v, tl, tm);
        build(2*v + 1, tm+1, tr);
        tree[v] = join(tree[v*2], tree[2*v + 1]);
    }
}

//update segtree
void update(int v, int tl, int tr, int pos, int new_val){
    if(tl == tr){
```

```
        tree[v] = new_val;
    }
    else{
        int tm = (tl + tr)/2;
        if(pos <= tm){
            update(2*v,tl,tm,pos,new_val);
        }
        else{
            update(2*v + 1,tm+1,tr,pos,new_val);
        }
        tree[v] = join(tree[v*2], tree[2*v + 1]);
    }
}

int query(int v, int L, int R, int l, int r){
    if(R < l || L > r){
        return 0;
    }
    if(L <= l && r <= R){
        return tree[v];
    }
    int m = (l + r)/2;
    return join(query(2*v, L, R, l, m), query(2*v + 1, L, R,
         m+1 ,r));
}
```

```
}

int main(){
    ios::sync_with_stdio(false);cin.tie(0);
    int n;
    cin>>n;
    tree = vector<int> (4*n);
    vec = vector<int> (n);
    for(int i=0;i<n;i++){
        cin>>vec[i];
    }
    build(1, 0, n-1);
    return 0;
}
```

10.2 2-fenwickTree

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
```

```
struct BIT {
    int n;
    vector<ll> bit;
    // Inicializa a BIT com tamanho n (1-based)
    BIT(int n) : n(n), bit(n + 1, 0) {}
    // Soma 'val' na posicao 'idx'
    void add(int idx, ll val) {
        for (; idx <= n; idx += idx & -idx)
            bit[idx] += val;
    }
    // Retorna a soma do prefixo [1, idx]
    ll query(int idx) {
        ll sum = 0;
        for (; idx > 0; idx -= idx & -idx)
            sum += bit[idx];
        return sum;
    }
    // Retorna a soma no intervalo [l, r]
    ll query(int l, int r) {
        return query(r) - query(l - 1);
    }
};
```